



Kisi-kisi Lomba Kompetensi Siswa Nasional 2026

SMA/MA/SMK/MAK/Sederajat



Cabang Ajang

**Teknologi Web
(Web Technologies)**

KISI-KISI

TEKNOLOGI WEB (*WEB TECHNOLOGIES*)



LKS DIKMEN
TINGKAT NASIONAL

**LOMBA KOMPETENSI SISWA (LKS) PENDIDIKAN MENENGAH
TINGKAT NASIONAL XXXIV
TAHUN 2026**

CLIENT-SIDE MODULE

CONTENTS

This module has the following files:

1. CLIENT SIDE MODULE.docx
2. CLIENT SIDE MODULE.pdf
3. CLIENT SIDE MODULE MEDIA.zip

PROJECT OVERVIEW

A client needs a visual map that can display various locations and routes to each location. This map should be interactive, allowing users to zoom, pan, and click on markers to view detailed information. The website needs to be compatible with modern web browsers and mobile devices.

This module is a 3-hour competition task. You are required to develop a web-based application using only HTML, CSS, and JavaScript. The core functionality revolves around an interactive map of Indonesia, allowing users to add, connect, and manage pinpoint locations, as well as find routes between them. You are allowed to improve the usability and user interface for a better user experience.

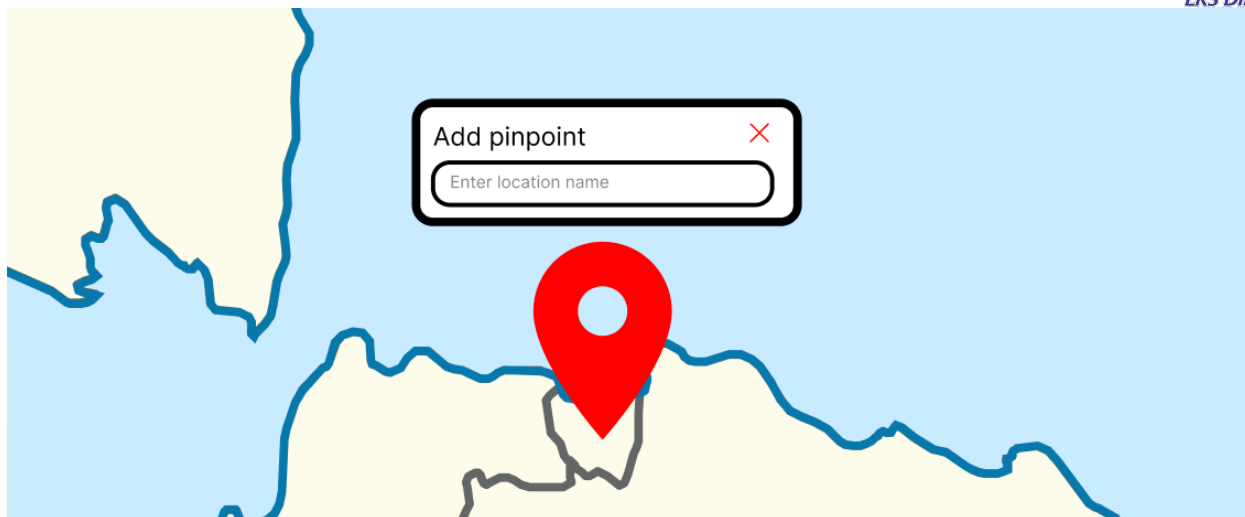
Loading the Map

You are provided with an SVG image of the Indonesia map. The map should be loaded into the page full size zoomed-in without leaving any blank space.



Adding Pinpoint Location

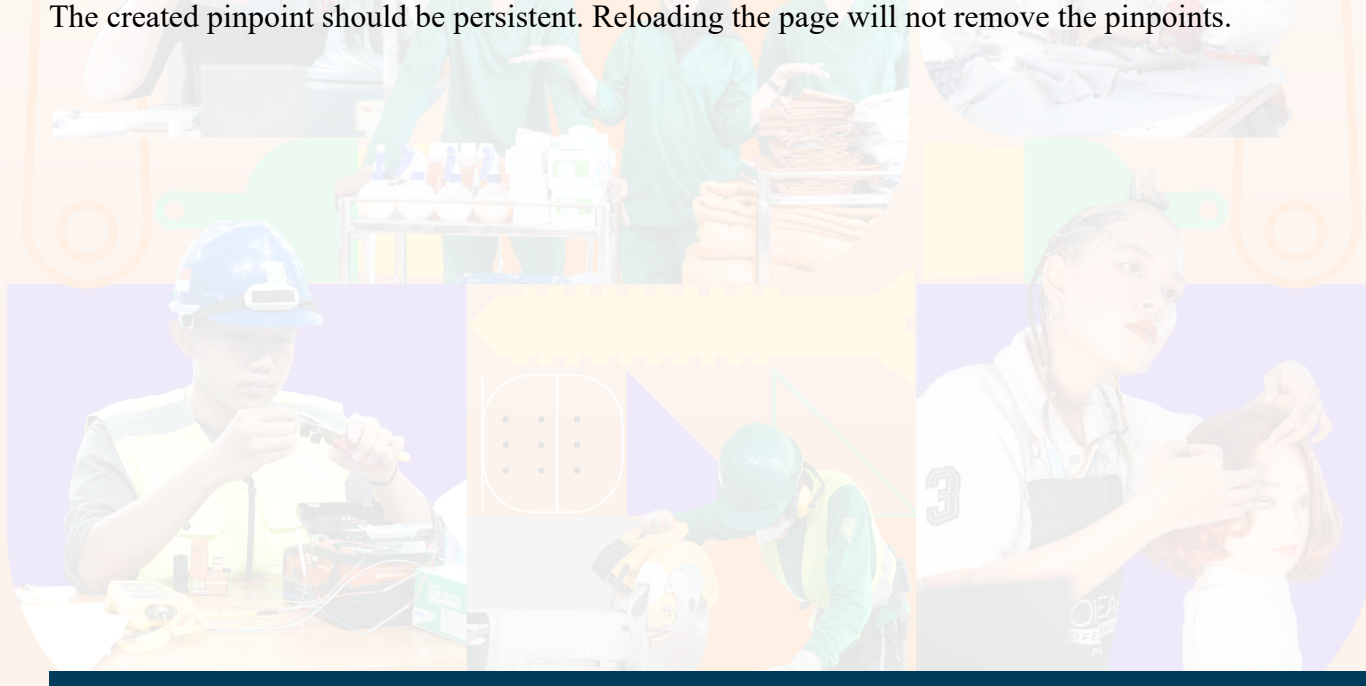
Double-click in the map will show a popup containing an input for location name to add a pinpoint in the map. Clicking the enter button will submit the form.



If the pinpoint is added, it should show the pinpoint name, a button to connect the pinpoint, and a button to delete it. A pinpoint should be marked with a red map-pin icon.

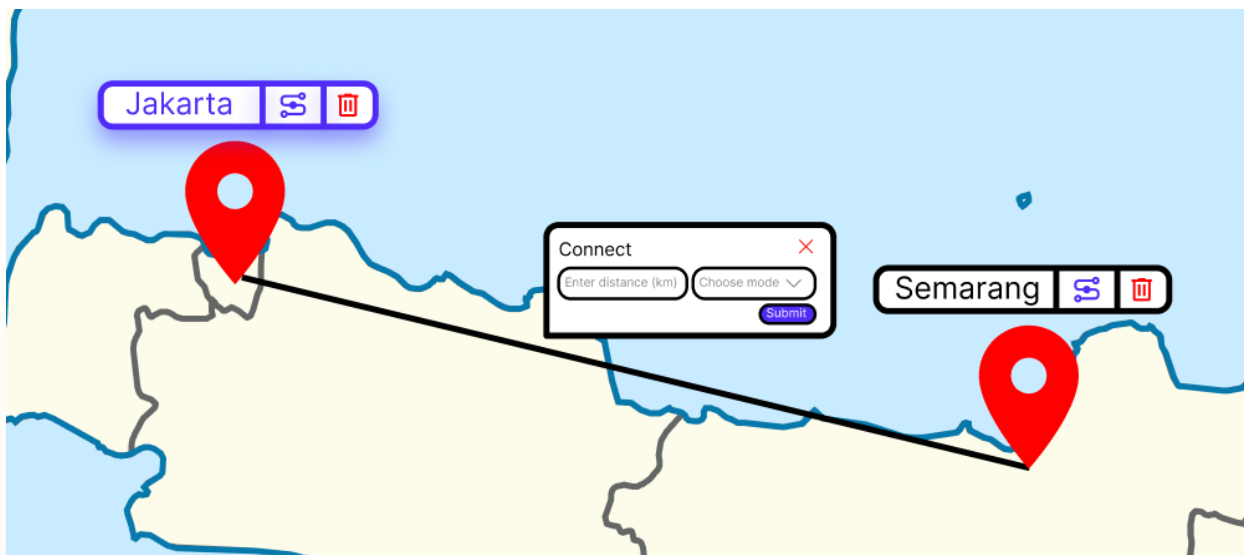


The created pinpoint should be persistent. Reloading the page will not remove the pinpoints.



Connect Locations

If the user clicked the connect icon, the pinpoint label shows signs that the pinpoint is in the state of connecting (e.g. glow, changed colors). Then, click the target pinpoint and the *connect popup* will show.



In this form, users need to enter the distance in kilometers and the transportation mode. The transportation modes consist of:

1. Train

- Line color: #33E339
- Speed: 120 km/h
- Cost: Rp500/km

2. Bus

- Line color: #A83BE8
- Speed: 80 km/h
- Cost: Rp100/km

3. Airplane

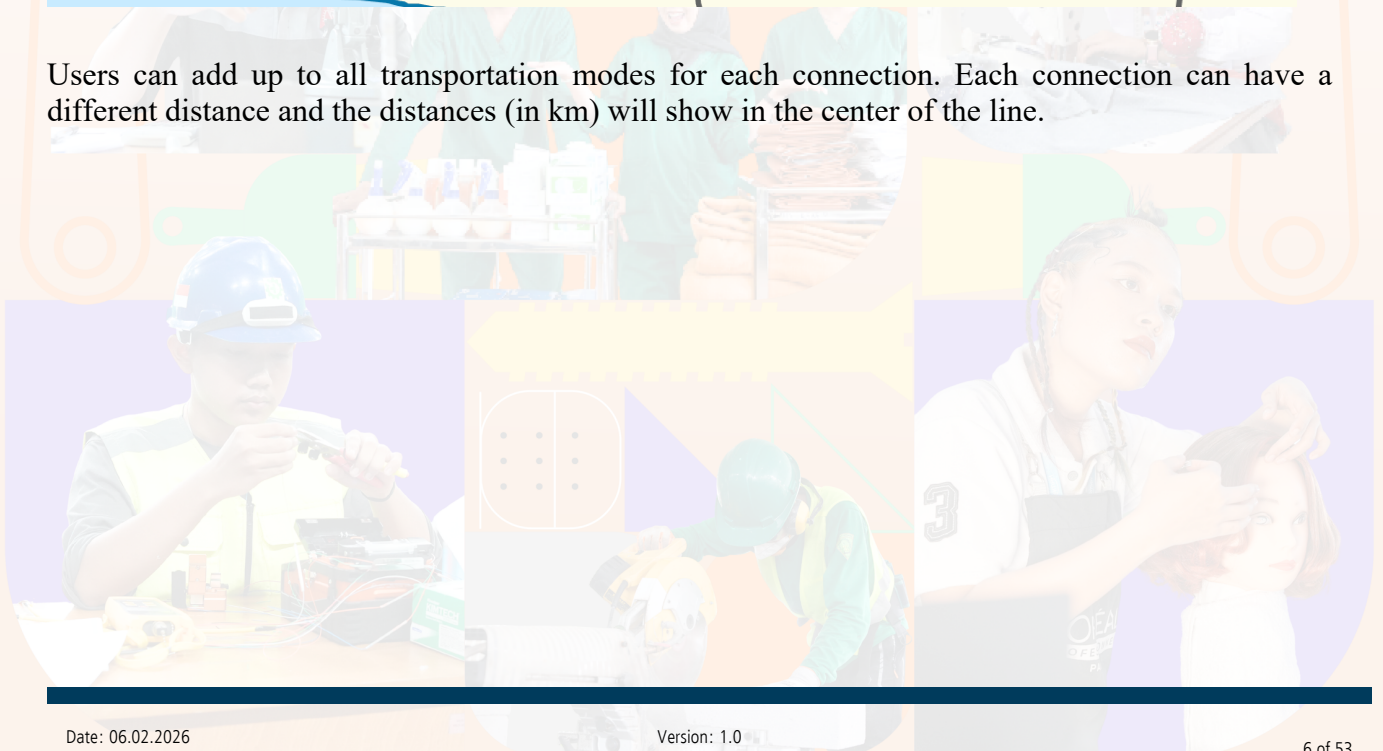
- Line color: #000000
- Speed: 800 km/h
- Cost: Rp1.000/km



After submit, a line will appear connecting both pinpoint locations with the distance present in the middle of the line.



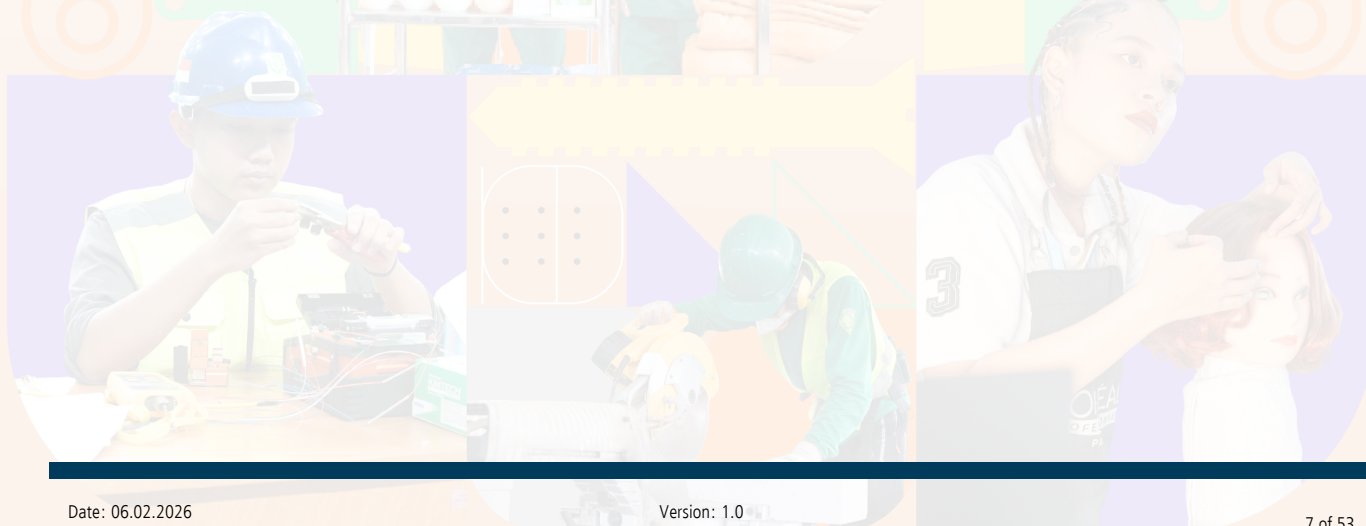
Users can add up to all transportation modes for each connection. Each connection can have a different distance and the distances (in km) will show in the center of the line.





Pan and zoom should be working correctly. Users should be able to zoom-in and zoom-out the map by clicking **CTRL+Scroll** or **CTRL+(+)** and **CTRL+(-)**. Zoom-in and zoom-out functionality should follow the cursor position.

In a zoomed map, users should be able to **move the map left and right by holding click and moving the cursor**.



Find Route

The Find Route modal should always exist on the left side of the browser viewport. Users should be able to input “*from pinpoint*” and “*to pinpoint*” with the exact keyword name. If one of the inputs contains invalid pinpoint name, **the search button should be disabled**.

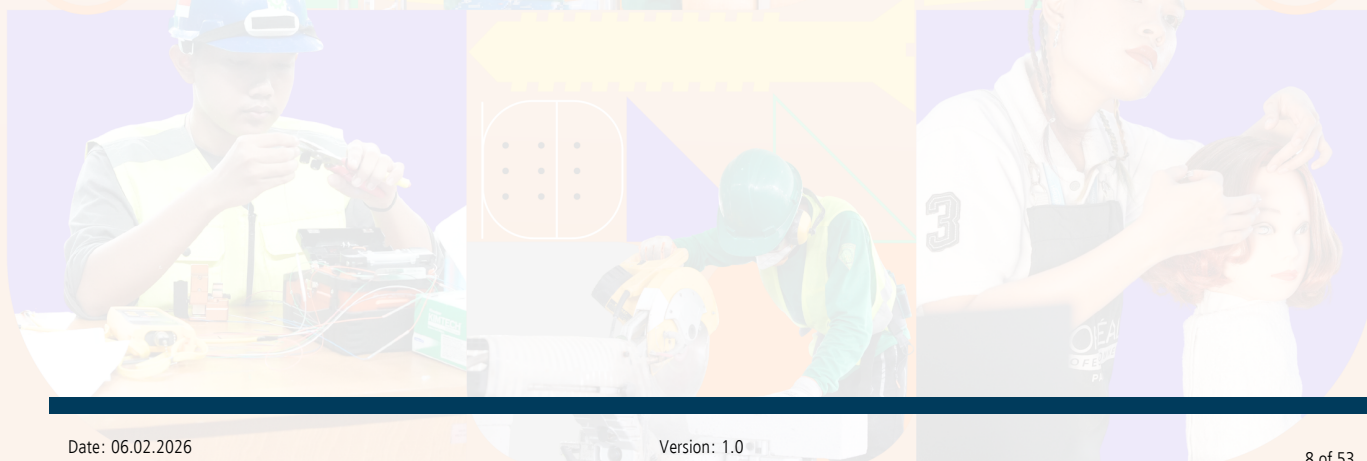
Clicking the search button will show **up to 10 possible routes** maximum, sorted by **fastest duration** or the **cheapest cost**. The order should be **the fastest** by default.

In each route, **show the route name, the transportation steps, the duration, and the total cost**.



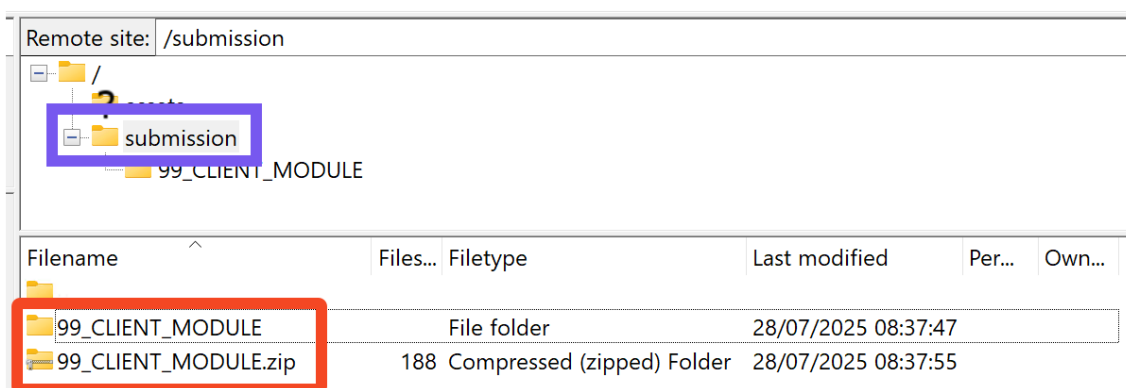
Removing Pinpoint and Connection

To remove a connection between pinpoints, the **user should be able to click the line, and press the del/backspace button**. A pinpoint can be removed by clicking the trash can button in the label, removing all of its connection.



INSTRUCTIONS TO THE COMPETITORS

1. This project will be assessed with either Firefox Developer Edition or Google Chrome web browser.
2. You may provide a README file for executing guide if necessary.
3. Create a root folder called **XX_CLIENT_SIDE_MODULE** in your local computer, where **XX** is your **computer number**.
4. Place your works inside the root folder. Make sure your works are working well when directly opening the **index.html** file.
5. Upload your **XX_CLIENT_SIDE_MODULE** to the FTP server inside “**submission**” folder (Please look at the example below).
6. Zip your root folder **XX_CLIENT_SIDE_MODULE** and upload to FTP server (Please look at the example below).



OTHERS

#	Sub-Criteria	Marks
1	Map functionalities	8.5
2	Pinpoint control	4.5
3	Pinpoint connections	7
4	Find Routes	10
Total		30

MOBILE WEB SERVICE MODULE

CONTENTS

This module has the following files:

1. MOBILE WEB SERVICE MODULE.docx
2. MOBILE WEB SERVICE MODULE.pdf
3. MOBILE WEB SERVICE MODULE MEDIA.zip

INTRODUCTION

The client has requested the development of a fast, mobile-first web app called **ApaKabar**, an **Online Media Platform**. The primary goal is to deliver a smooth and accessible experience, especially for users on mobile devices. Performance and ease of access are top priorities, as the main target audience consists of mobile users.

PROJECT OVERVIEW

Your task is to build a **Mobile-first Web Application** using a JavaScript framework of your choice either **React or Vue**. Alternatively, you may build it using **Vanilla JavaScript (Native)** without any framework. A **pre-built API is provided** to support your development. These are the following core features:

- Fully optimized for mobile screens only
- Home page with personalized news based on selected categories
- Discover page with search, pagination and category filters
- The article detail page displays all attributes of the selected article.
- Bookmark feature for saving articles
- Light, dark, and system appearance modes

MOBILE WEB LAYOUT

Screen Size: Max width of 480px (mobile screen only).

The layout consist of three main sections:

- Header Bar (Top)
- Main Content (Scrollable)
- Navigation Bar (Bottom)

The header bar is fixed at the top of the screen which displays the current page title and an optional back button for navigation.

The main content is positioned between the header and navigation bars. It must be scrollable when content exceeds the viewport height.

The navigation bar is fixed at the bottom of the screen.

NAVIGATION BAR

There are 4 buttons in the navigation bar:

1. Home
2. Discover
3. Bookmark
4. Settings

THE DATA API

API Endpoint provided on <http://api-mobile-service.lksn.id>

The API includes the following main endpoints:

- Get all posts
 - GET /api/v1/posts
- Get a post by slug
 - GET /api/v1/posts/:slug
- Get all categories
 - GET /api/v1/categories

QUERYING POSTS

The /api/v1/posts endpoint supports several optional query parameters to filter and paginate the results:

- page — number, min: 1, default: 1 (e.g. 5 for page 5)
- per_page — number, min: 1, default: 20 (e.g. 50 to show 50 posts)
- order_by — latest or popular, default: latest
- search — string, default: empty (""), to reset search
- category — comma-separated category slugs (e.g. slug_1,slug_2)

HOME PAGE

The first button in the navigation bar opens the Home page.

BREAKING NEWS

This section highlights the urgent news articles. Breaking news items are shown in a horizontally scrollable list with scroll snapping, allowing users to swipe through stories smoothly one by one.

RECOMMENDATION LIST

These recommendations are articles based on the user's selected category preferences and aim to show content that matches their interests.

DISCOVER PAGE

The second button in the navigation bar opens the Discover page.
This page allows users to explore news articles.

SEARCH

Users can search for articles using keywords. The search input is designed to improve performance and reduce unnecessary API calls by using a debounce mechanism.

- The app waits until the user has stopped typing for 0.5–1 second before sending a request to the API.
- It means that search does **not** trigger an API request on every keystroke or on clicking any button.

FILTER BY CATEGORY

The category filter allows users to filter news articles based on specific categories. The list of categories is fetched dynamically from the API.

INFINITE SCROLL

When the user scrolls to the bottom, the web page should load further article data and display further articles to the users. This is also known as infinite scrolling.

Please fine-tune the timing and smoothness of infinite scrolling. The loading of further content should be not too late, and not loading too early. In other words, the infinite scroll should not feel laggy, and should not eagerly load too much data. Please ensure the data loading is correct. That means there should not have multiple instances of the same record displayed, and not missing any records to display.

ARTICLE DETAIL PAGE

The Article Detail Page displays the full content of a selected news article.
It shows all relevant information, including:

- Category name
- Title
- Publish date
- Author name
- Visited count
- Thumbnail or cover image
- Full content or body text
- Tags

At the bottom of the page, there should be a section that displays 3 related news articles that from the same category as the current article and exclude the article currently being viewed

BOOKMARK

The third button in the navigation bar opens the Bookmark view.

This page displays a list of articles that the user has saved for later reading.

Users can add or remove bookmarks directly from the article detail page or from the article list view.

SETTINGS

The fourth button in the navigation bar opens the Settings view.

THEME MODE

Users can choose between three theme options: light, dark and system (follow the device's light/dark setting)

CATEGORY PREFERENCE

Users can select or unselect their preferred news categories. These preferences will be used to personalize the Recommendation section on the Home page.

OFFLINE MODE

When users lose internet connection while using the app, a dedicated offline page (game.html) is displayed to maintain a user experience. The html file is provided in media files.

MEDIA FILES

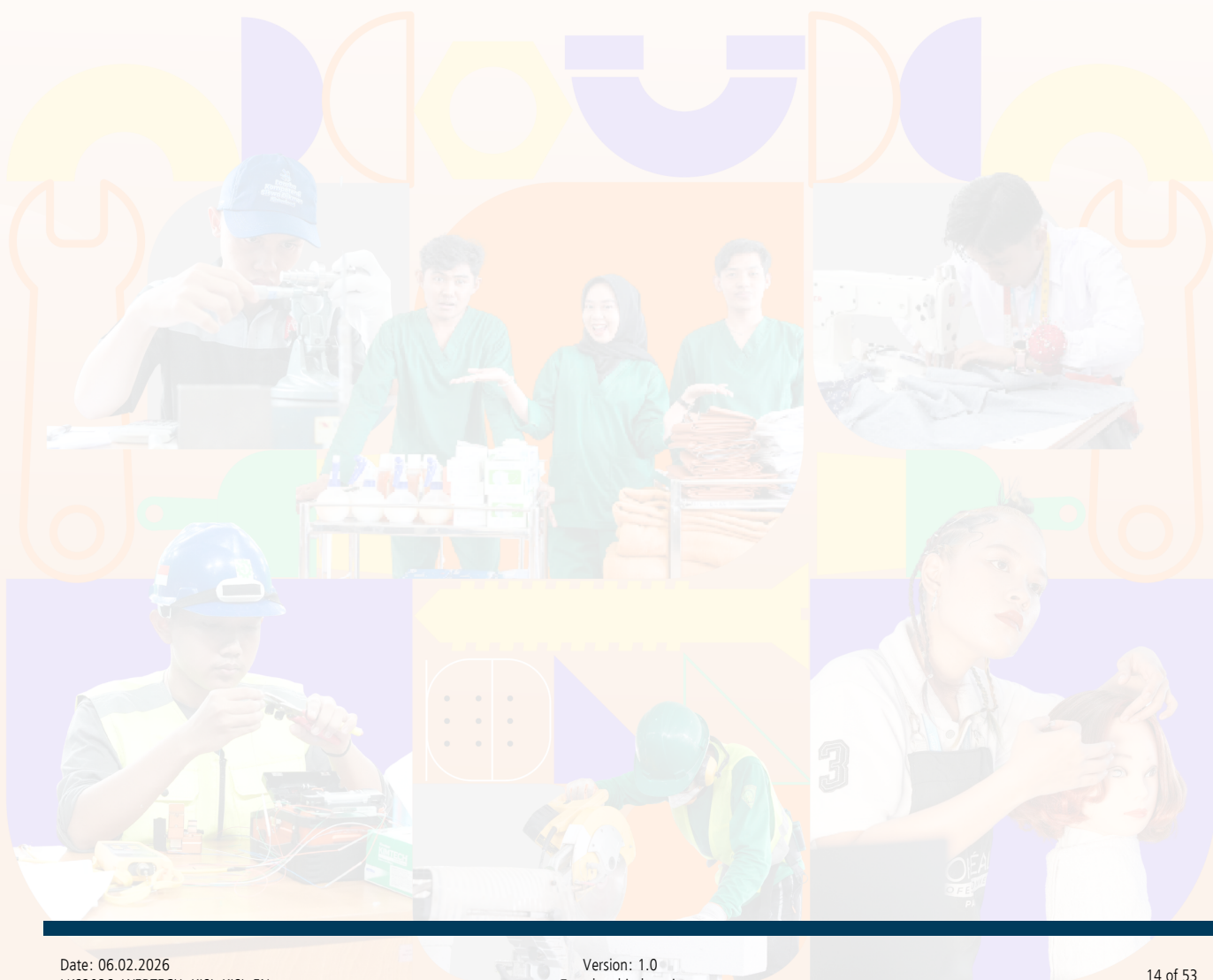
- Wireframes – visual guides for layout and UI reference
- JavaScript Frameworks – React/Vue (with Vite setup)
- HTML file for offline page

INSTRUCTIONS FOR COMPETITORS

- You must configure the API base URL in a single place, such as:
 - Using an environment variable:
 - `VITE_API_URL=http://api-mobile-service.lksn.id` (in a .env file)
 - Or defining a constant in your code:
 - `const apiUrl = 'http://api-mobile-service.lksn.id'`
- Please configure the **manifest.json** and essential meta tags for both Android and iOS mobile web systems.
- The viewport should also be configured for mobile web friendly viewing.
- Please ensure having good accessibility. We will assess the Chrome Lighthouse accessibility score.
- Please also ensure creating easy-to-maintain JavaScript code.

PROJECT STRUCTURE & SUBMISSION

1. Rename project folder to: **XX_MOBILE_WEB_SERVICE_MODULE** where **XX** is your computer number.
2. If you are using NodeJs Framework, please place the **build version** of your project to **XX_MOBILE_WEB_SERVICE_MODULE**, so your project can be accessible from **index.html**
3. Zip the root folder/your project (not your build version) into a single file **XX_MOBILE_WEB_SERVICE_MODULE.zip**.
Exclude node_modules when zipping the project (if exists).
4. Upload your **XX_MOBILE_WEB_SERVICE_MODULE** and **XX_MOBILE_WEB_SERVICE_MODULE.zip** to the FTP server inside “**submission**” folder.



SERVER SIDE MODULE

CONTENTS

This module has the following files:

1. SERVER MODULE PHASE 1.docx
2. SERVER MODULE PHASE 1.pdf
3. SERVER MODULE PHASE 1 MEDIA.zip

INTRODUCTION

The client has approached you to build a backend system and frontend web application for a **Financial Management Application** called **PintarMenabung**. The system must allow users to register, manage their wallets, record income and expenses, and view basic financial reports.

PROJECT OVERVIEW

This project consists of two main phases: In Phase 1, you are required to develop a **REST API** using the **Laravel Framework**, implementing core features such as user authentication, wallet management, transaction recording and financial reporting. In Phase 2, you will build a **frontend web application** that integrates with the API, using either **React** or **Vue** as your framework of choice.

To support your work, media files have been provided, including Postman Collection and Environment, a Database Dump (.sql), and Framework Scaffolding.

PHASE 1 - Backend Development (REST API)

A. Authentication

There are three endpoints: register, login, and logout. **Sanctum** should be used to create tokens. The generated token will be used to access all available endpoints. The logout function should only deactivate the token from the specific device making the request.

A1: Register

POST /api/auth/register

For users to create an account in the application.

Request Headers

Accept	application/json
--------	------------------

Body (JSON)

full_name	required.
email	required, valid email format, unique.
password	required, min 6 chars.

A1a: Response Success

Status Code	201
Body	<pre>{ "status": "success", "message": "Registration successful", "data": { "name": "Dedi", "email": "dedi@webtech.id", "updated_at": "2025-07-11T16:01:46.000000Z", "created_at": "2025-07-11T16:01:46.000000Z", "id": 6, "token": "1 wwwOrgoVe4Xyj5dvUmnf6WTPTaA..." } }</pre>

A1b: Response Invalid Field

Status Code	422
Body	<pre>{ "status": "error", "message": "Invalid field", "errors": { "name": ["The name field is required."], "email": ["The email has already been taken."] } }</pre>

```

    ],
    "password": [
        "The password field must be at least 6 characters."
    ]
}
}

```

A2: Login

POST /api/auth/login

For users login into the application using email and password.

Request Headers

Accept	application/json
--------	------------------

Body (JSON)

email	required.
password	required.

A2a: Response Success

Status Code	200
Body	<pre> { "status": "success", "message": "Login successful", "data": { </pre>

	<pre> "id": 1, "name": "Budi", "email": "budi@webtech.id", "created_at": "2025-07-11T16:01:38.000000Z", "updated_at": "2025-07-11T16:01:38.000000Z", "token": "2 0Q2TAdua0FOslp1F4cxfrzQgkp..." } } </pre>
--	--

A2b: Response Failed

Status Code	401
Body	<pre> { "status": "error", "message": "Username or password incorrect" } </pre>

A3: Logout

POST /api/auth/logout

For logged-in user logout from the application. It specifically deactivates the authentication token associated with the current device making the request.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

A3a: Response Success

Status Code	200
Body	<pre>{ "status": "success", "message": "Logout successful" }</pre>

A3b: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

B. Currency and Category

The application uses a Category system to classify financial transactions. Each category must be assigned a type, which can either be:

- Income – for transactions that increase the wallet balance (e.g., salary, bonus)
- Expense – for transactions that decrease the wallet balance (e.g., groceries, transportation)

Categories help users organize their finances and are required when creating a transaction.

B1: Get All Currencies

GET /api/currencies

For logged-in users to get all available currencies.

Request Headers

Accept	application/json
--------	------------------

Authorization	Bearer <TOKEN>
---------------	----------------

B1a: Response Success

Status Code	200
Body	<pre>{ "message": "Get all currencies successful", "data": { "currencies": [{ "id": 1, "name": "US Dollar", "symbol": "\$", "code": "USD", "created_at": "2025-07-11T16:01:37...", "updated_at": "2025-07-11T16:01:37..." }, ...] } }</pre>

B1b: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

--	--

B2: Get All Categories

GET /api/categories

For logged-in users to get all available categories.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

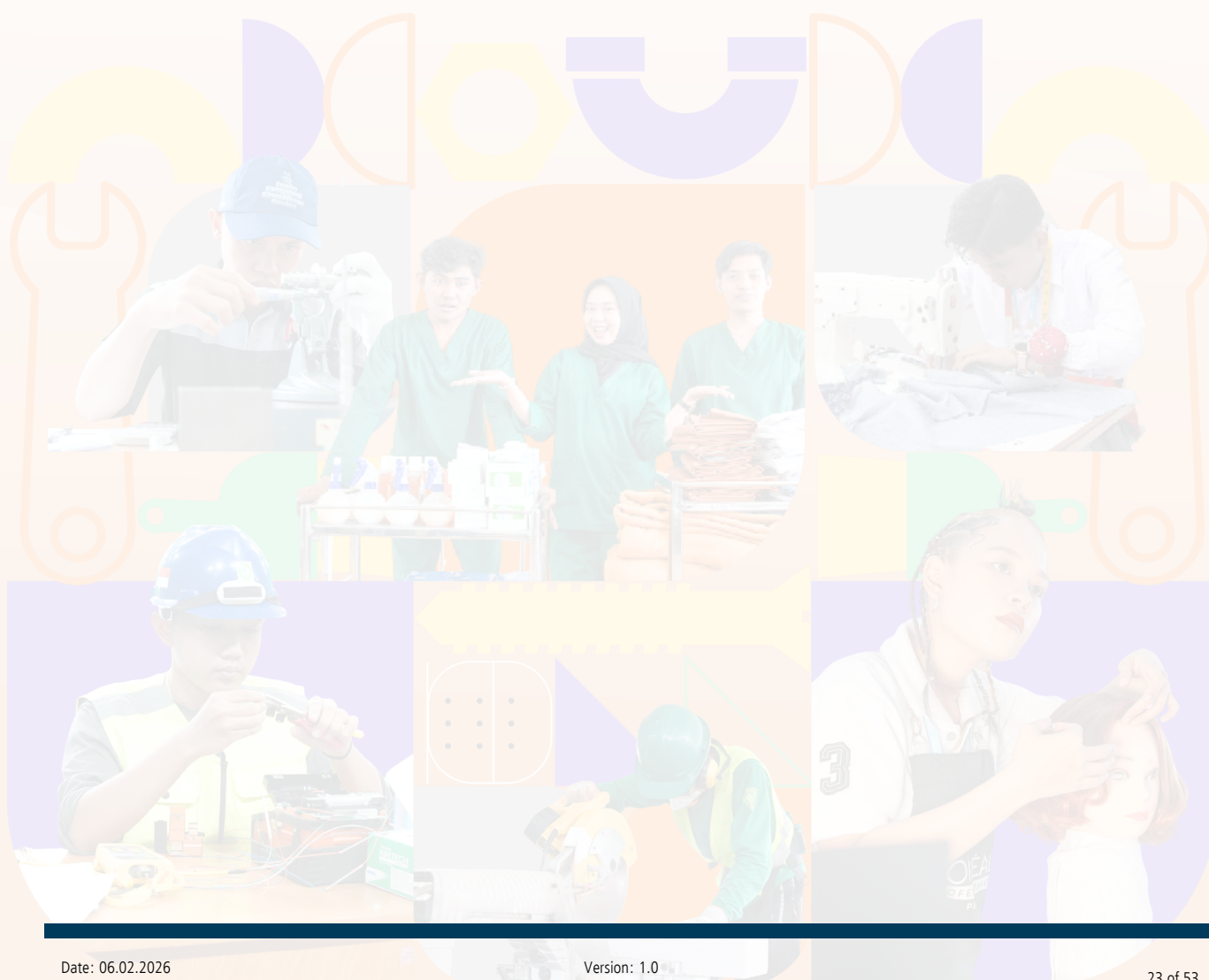
B2a: Response Success

Status Code	200
Body	<pre>{ "status": "success", "message": "Get all categories successful", "data": { "categories": [{ "id": 1, "name": "Outgoing Transfer", "icon": "📶", "type": "EXPENSE", "created_at": "2025-07-11T16:01:38...", "updated_at": "2025-07-11T16:01:38..." }, ...] } }</pre>

	<pre> } } </pre>
--	------------------

B2b: Response Invalid Token

Status Code	401
Body	<pre> { "status": "error", "message": "Unauthenticated." } </pre>



C. Wallet

A Wallet is a financial container used to track a user's income and expenses. Users can create multiple wallets to organize their finances (e.g., personal, savings, or business wallets).

Each wallet must display a current balance, which is automatically and accurately calculated based on all associated transactions.

- Income transactions will increase the balance.
- Expense transactions will decrease the balance.

The system must ensure that the wallet balance is always up-to-date and reflects the correct total from all transactions made within that wallet.

C1: Add Wallet

POST /api/wallets

For logged-in users to add a wallet.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Body (JSON)

name	required.
currency_code	required, valid currency_code data.

C1a: Response Success

Status Code	201
Body	<pre>{ "status": "success", "message": "Wallet added successful", "data": { "name": "Savings", "user_id": 1, "updated_at": "2025-07-11T16:22:44.000000Z", "created_at": "2025-07-11T16:22:44.000000Z", "id": 26, "currency_code": "IDR" } }</pre>

C1b: Response Invalid Field

Status Code	422
Body	<pre>{ "status": "error", "message": "Invalid field", "errors": { "name": ["The name field is required."], "currency_code": ["The selected currency code is invalid."] } }</pre>

	<pre>] } } </pre>
--	--------------------

C1c: Response Invalid Token

Status Code	401
Body	<pre> { "status": "error", "message": "Unauthenticated." } </pre>

C2: Update Wallet

PUT /api/wallets/:walletId

For logged-in users to update their own wallet.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Body (JSON)

name	required.
------	-----------

C2a: Response Success

Status Code	200
Body	<pre>{ "status": "success", "message": "Wallet updated successful", "data": { "id": 1, "user_id": 1, "name": "Cash IDR", "created_at": "2025-07-11T16:01:38.000000Z", "updated_at": "2025-07-11T16:26:25.000000Z", "deleted_at": null, "currency_code": "IDR" } }</pre>

C2b: Response Not Found

Status Code	404
Body	<pre>{ "status": "error", "message": "Not found" }</pre>

C2c: Response Forbidden

Status Code	403
Body	<pre>{ "status": "error", "message": "Forbidden access" }</pre>

C2d: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

C3: Delete Wallet

DELETE /api/wallets/:walletId

For logged-in users to delete their own wallet.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

C3a: Response Success

Status Code	200
Body	<pre>{ "status": "success", "message": "Wallet deleted successful" }</pre>

C2b: Response Not Found

Status Code	404
Body	<pre>{ "status": "error", "message": "Not found" }</pre>

C2c: Response Forbidden

Status Code	403
Body	<pre>{ "status": "error", "message": "Forbidden access" }</pre>

C2d: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

C4: Get All Wallets

GET /api/wallets

For logged-in users to get all of their own wallets.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

C4a: Response Success

Status Code	200
Body	<pre>{ "status": "success", "message": "Get all wallets successful", "data": { "wallets": [{ "id": 2, "user_id": 1, "name": "Bank", "created_at": "2025-07-11T16:01:38...", "updated_at": "2025-07-11T16:01:38...", "deleted_at": null, "currency_code": "IDR", "balance": 12250000 }, ...] } }</pre>

C4b: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

C5: Get Detail Wallet

GET /api/wallets/:walletId

For logged-in users to get details about their own wallets.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

C5a: Response Success

Status Code	200
Body	<pre>{ "status": "success", "message": "Get detail wallet successful", "data": { "id": 1, "user_id": 1, "name": "Cash IDR", "created_at": "2025-07-11T16:01:38.000000Z", } }</pre>

	<pre> "updated_at": "2025-07-11T16:26:25.000000Z", "deleted_at": null, "currency_code": "IDR", "balance": 4865000 } </pre>
--	--

C5b: Response Forbidden

Status Code	403
Body	<pre> { "status": "error", "message": "Forbidden access" } </pre>

C5c: Response Not Found

Status Code	404
Body	<pre> { "status": "error", "message": "Not found" } </pre>

C5c: Response Forbidden

Status Code	403
Body	<pre> { </pre>

	<pre> "status": "error", "message": "Forbidden access" } </pre>
--	---

C5d: Response Invalid Token

Status Code	401
Body	<pre> { "status": "error", "message": "Unauthenticated." } </pre>

D. Transaction

A Transaction represents a financial record that occurs within a specific wallet. Each transaction must be linked to a wallet and assigned to a category, which determines whether it is an income or an expense.

Key attributes of a transaction include:

- Wallet – the wallet where the transaction belongs
- Category – defines the type (income or expense) based on the selected category
- Amount – the monetary value of the transaction (positive number)
- Date – the date when the transaction occurred
- Note (optional) – a short note about the transaction

The transaction's impact on the wallet balance depends on the type defined by the selected category:

- Income category → increases the wallet balance
- Expense category → decreases the wallet balance

The system must ensure that transactions are accurately recorded and that wallet balances are updated accordingly.

D1: Add Transaction

POST /api/transactions

For logged-in users to add a transaction.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Body (JSON)

wallet_id	required, valid wallet_id data.
category_id	required, valid category_id data.
amount	required, integer, greater or equal 1
date	required, date format “Y-m-d”
note	optional

D1a: Response Success

Status Code	201
Body	<pre>{ "status": "success", "message": "Transaction added successful", "data": { "category_id": 3, "wallet_id": 1, "amount": 50000, "note": "starbucks", "date": "2025-07-31", "updated_at": "2025-07-11T16:44:09.000000Z", "created_at": "2025-07-11T16:44:09.000000Z", "id": 842 } }</pre>

D1b: Response Invalid Field

Status Code	422
Body	<pre>{ "status": "error", "message": "Invalid field", "errors": { "name": ["The name field is required."], "currency_code": ["The selected currency code is invalid."] } }</pre>

D1c: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

D2: Delete Transaction

DELETE /api/transactions/:transactionId

For logged-in users to delete their own transaction.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

D2a: Response Success

Status Code	200
Body	<pre>{ "status": "success", "message": "Transaction deleted successful" }</pre>

D2b: Response Not Found

Status Code	404
Body	<pre>{ "status": "error", "message": "Not found" }</pre>

D2c: Response Forbidden

Status Code	403
Body	<pre>{ "status": "error", "message": "Forbidden access" }</pre>

D2d: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

D3: Get Transactions

GET /api/transactions

For logged-in users to get all of their own transactions ordered by transaction date descending.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Request Parameters

page	optional, integer, default 1
------	------------------------------

per_page	optional, integer, default 25
month	optional, integer, between 1 and 12
year	optional, integer

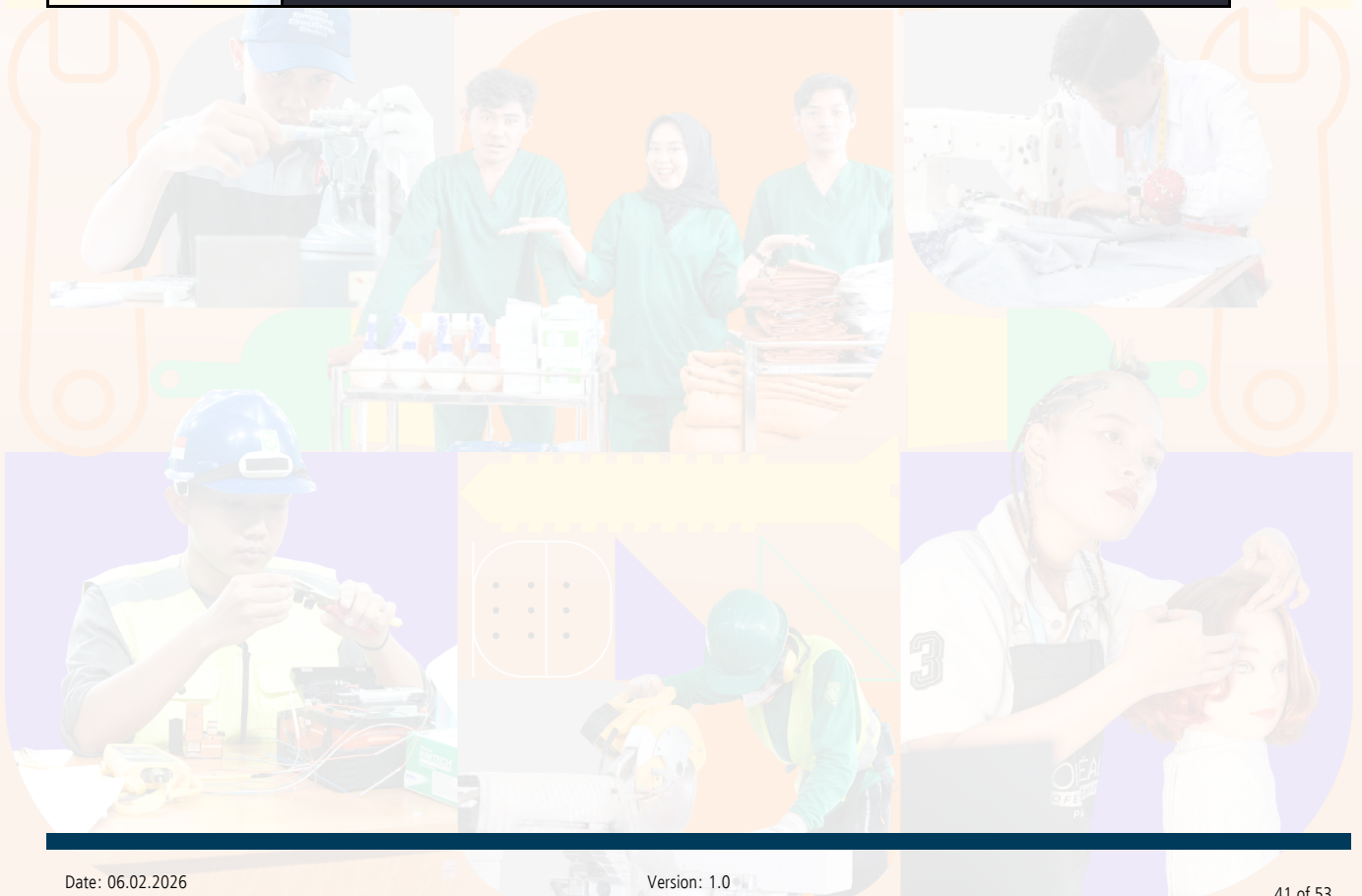
D3a: Response Success

Status Code	200
Body	<pre>{ "current_page": 1, "data": [{ "id": 842, "category_id": 3, "wallet_id": 1, "amount": 50000, "note": "starbucks", "date": "2025-07-31", "created_at": "2025-07-11T16:44:09.000000Z", "updated_at": "2025-07-11T16:44:09.000000Z", "wallet": { "id": 1, "user_id": 1, "name": "Savings IDR", "created_at": "2025-07-11T16:01:38.000000Z", "updated_at": "2025-07-11T16:26:25.000000Z", "deleted_at": null, "currency_code": "IDR" } }] }</pre>

```

"category": {
  "id": 3,
  "name": "Food & Drinks",
  "icon": "🍔",
  "type": "EXPENSE",
  "created_at": "2025-07-11T16:01:38.000000Z",
  "updated_at": "2025-07-11T16:01:38.000000Z"
},
...
],
"from": 1,
"last_page": 170,
"per_page": 1,
"to": 1,
"total": 170
}

```



D3b: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

E. Financial Report

E1: Get Summary by Expense Category

GET /api/reports/summary-by-category/expense

For logged-in users to get a summary by expense category.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Request Parameters

month	optional, integer, between 1 and 12
year	optional, integer

E1a: Response Success

Status Code	200
Body	<pre>{ "status": "success", "message": "Get summary by expense category successful", "data": { "summary": [{ "category": { "id": 13, "name": "Groceries", "icon": "🛒", "color": "#55EFC4", "type": "EXPENSE", "created_at": "2025-07-28T02:37:46.000000Z", "updated_at": "2025-07-28T02:37:46.000000Z" }, "amount": 50000 }, ...] } }</pre>

E1b: Response Invalid Token

Status Code	401
Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>

E2: Get Summary by Income Category

GET /api/reports/summary-by-category/income

For logged-in users to get a summary by income category.

Request Headers

Accept	application/json
Authorization	Bearer <TOKEN>

Request Parameters

month	optional, integer, between 1 and 12
year	optional, integer

E2a: Response Success

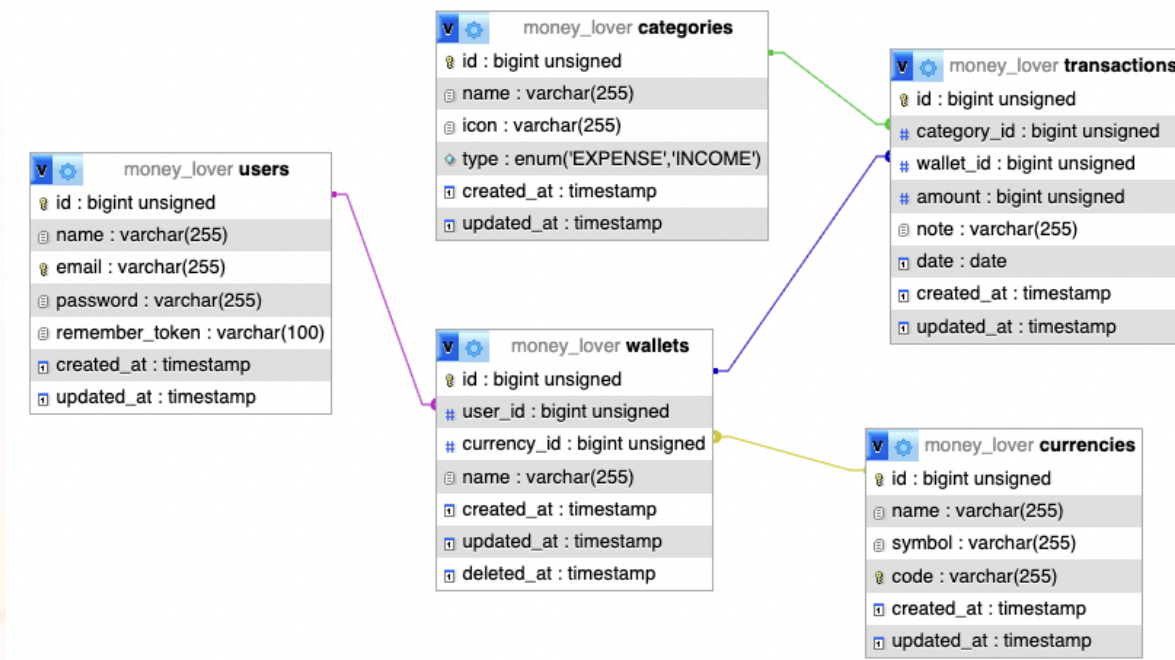
Status Code	200
Body	<pre>{ "status": "success", "message": "Get summary by income category successful", "data": { "summary": [{ "category": { "id": 2, "name": "Incoming Transfer", "icon": "📶", "color": "#00B894", "type": "INCOME", "created_at": "2025-07-28T02:37:46.000000Z", "updated_at": "2025-07-28T02:37:46.000000Z" }, "amount": 1050000 }, ...] } }</pre>

E2b: Response Invalid Token

Status Code	401
-------------	-----

Body	<pre>{ "status": "error", "message": "Unauthenticated." }</pre>
------	---

ER DIAGRAM



MEDIA FILES

- Postman Collection & Environment – contains all required API endpoints
- Database Dump (.sql) – includes initial schema and sample data
- Framework Installers – Laravel

INSTRUCTIONS

- You are not allowed to create additional endpoints.
- You can add extra fields to the response body if needed, **just make sure all required fields are included and the response structure is correct.**

PROJECT STRUCTURE & SUBMISSION

1. Create a root folder named **XX_SERVER_MODULE** where **XX** is your computer number.
2. Inside the root folder, organize your files as follows:
 - **XX_SERVER_MODULE/**
 - **BACKEND/** # Your backend project code
 - **db-dump.sql** # Exported database dump
 - **db-diagram.pdf** # ER Diagram
3. Zip the root folder into a single file:
XX_SERVER_MODULE_PHASE_1.zip
4. Exclude the following folders when zipping:
 - **vendor/**
 - **node_modules/**
5. Submit **XX_SERVER_MODULE_PHASE_1.zip** to the submission page.

SERVER SIDE MODULE

CONTENTS

This module has the following files:

1. SERVER MODULE PHASE 2.docx
2. SERVER MODULE PHASE 2.pdf
3. SERVER MODULE PHASE 2 MEDIA.zip

INTRODUCTION

The client has approached you to build a backend system and frontend web application for a **Financial Management Application** called **PintarMenabung**. The system must allow users to register, manage their wallets, record income and expenses, and view basic financial reports.

API Configuration

API URL: <http://XX.api-server-module.lksn.id/> (replace XX with your computer number)

PROJECT OVERVIEW

This project consists of two main phases: In Phase 1, you are required to develop a **REST API** using the **Laravel Framework**, implementing core features such as user authentication, wallet management, transaction recording and financial reporting. In Phase 2, you will build a **frontend web application** that integrates with the API, using either **React** or **Vue** as your framework of choice.

To support your work, media files have been provided, including Postman Collection and Environment, a Database Dump (.sql), and Framework Scaffolding.

PHASE 2 - Frontend Web Application

Pages	Features
General	- The document title should reflect the current page (e.g., "Login", "Register", "Overview"). - After a successful login, display a Logout button in the navbar. - Users can log out by clicking the Logout button in the navbar.
Register	Path: /register - Users can register using the following fields: - Full Name

	○ Email ○ Password • Display specific error messages based on the relevant input field when registration fails. • After a successful registration, users are redirected to the Overview page. • If a logged-in user accesses this page, they will be redirected to the Overview page automatically.
Login	Path: /login • Users can log in using their existing credentials (email and password). • Display specific error messages when login fails, based on the relevant input field. • After a successful login, users are redirected to the Overview page. • If a logged-in user accesses this page, they will be redirected to the Overview page automatically.
Overview Page	Path: / Access: Only accessible to logged-in users Displayed Information • User Wallets ○ Show each wallet's name and current balance • Recent Transactions (from all wallets) ○ Transaction Date ▪ Format: Human-readable (e.g., “Jun 7, 2025”) ▪ Only show the date once per group; hide if it's the same as the previous transaction's date. ○ Category Icon ▪ Red for expense ▪ Green for income ○ Category Name ○ Wallet Name ○ Amount ▪ Prefix with - for expenses ▪ Prefix with + for income

	▪ Display formatted as currency (e.g., “IDR 75.000” or “USD 12”) ○ Note (if provided) Actions • Add Wallet ○ Click the “+” button under the Balance section ○ Fill out the wallet form and click “Save” • Add Transaction ○ Click the “Add Transaction” button ○ Fill out the transaction form and click “Save” • Delete Transaction ○ Double-click on a transaction item ○ Confirm the deletion in the popup dialog
Detail Wallet	Path: /wallets/:walletId Access: Only accessible to logged-in users Navigation • Users can access this page by clicking a wallet item on the Overview Page. Displayed Information • Wallet Information ○ Display the wallet name and current balance. • Transactions ○ Show wallet transactions based on the selected month and year. ○ Year selection range: from 2015 to 2030. ○ Refer to the Overview Page for transaction details/attributes. • Summary of expense and income per month ○ Display in doughnut chart ○ Data should be fetch from API ○ Labels should be visible in the bottom Actions • Edit Wallet Name ○ Click the Wallet Name. ○ Rename the wallet name. ○ Press “Enter” to save. • Delete Wallet ○ Click the Wallet Name. ○ Empty the form. ○ Press “Enter” to show the confirmation dialog and click “Ok” to delete. • Add Transaction ○ Click the "Add Transaction" button.

	○ The wallet field in the form should be auto-selected based on the current wallet. ○ Fill in the form and click "Save". • Delete Transaction ○ Double-click on a transaction item ○ Confirm the deletion in the popup dialog • Transfer Between Wallets ○ Click the "Transfer Money" button. ○ The source wallet should be auto-selected based on the current wallet. ○ Fill in the form and click "Transfer".
--	--

Shortcuts

Keymap	Description
Alt + W	Open "Add Wallet" form
Alt + N	Open "Add Transaction" form
Alt + T	Open "Transfer Money" form
Esc	Close the form

MEDIA FILES

- HTML Template – to be used as the base layout for the frontend.
- Utility Scripts – helper functions for currency, date formatting, etc.
- Chart-js example code for react and vue.

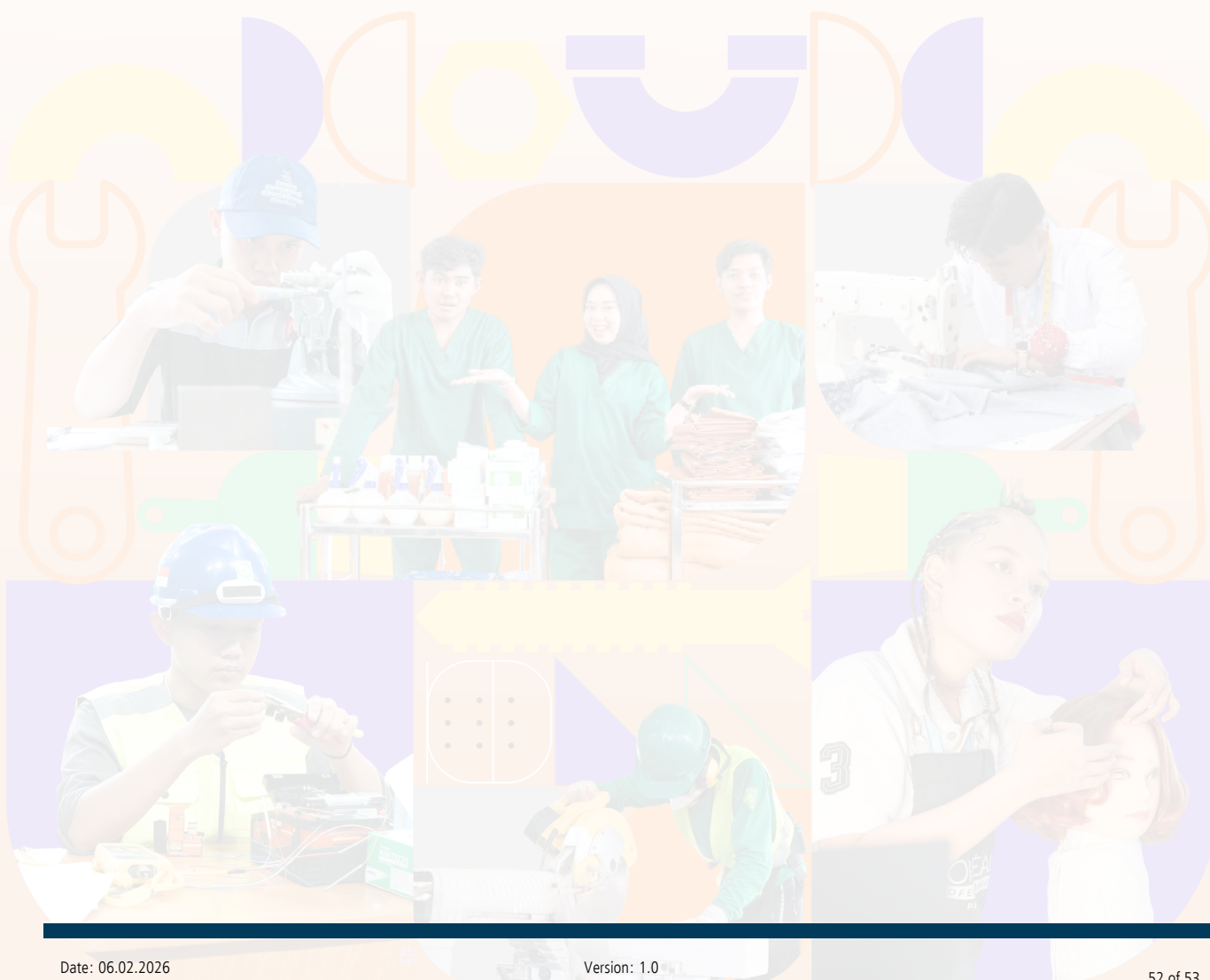
INSTRUCTIONS

- You are not allowed to create additional endpoints.
- You can add extra fields to the response body if needed, **just make sure all required fields are included and the response structure is correct.**

PROJECT STRUCTURE & SUBMISSION

1. Create a root folder named **XX_SERVER_MODULE_PHASE_2** where **XX** is your computer number.
2. Inside the root folder, organize your files as follows:
 - **XX_SERVER_MODULE_PHASE_2/**
 - **public/**

- *src/*
 - ...
 - *package.json*
3. Zip the root folder into a single file:
XX_SERVER_MODULE_PHASE_2.zip
 4. Exclude the following folders when zipping:
 - *vendor/*
 - *node_modules/*
 5. Submit **XX_SERVER_MODULE_PHASE_2.zip** to the submission page.





Kemendikdasmen
Pusat Prestasi Nasional

Jl. Gardu, Srengseng Sawah, Kec. Jagakarsa, Kota Jakarta Selatan, Daerah Khusus Ibukota Jakarta.
Unit Layanan Tanya Prestasi ☎ 0852 8277 7740

📷 📺 📢 puspresnas 📺 📢 Pusat Prestasi Nasional 🌐 pusatprestasinasional.kemendikdasmen.go.id